CDCT because they're becoming such a key part of how they do integration testing.

Even though many find CDCT helpful, it's not a perfect solution, and some people face difficulties when trying to make it work well at a larger scale. The PactFlow team recognized these challenges and looked for ways to address them. Some of the problems they wanted to solve included the following:

- The need for a lot of technical knowledge and effort
- Limitations in who can use it (mostly those who work directly with the code)
- Its compatibility with specific types of systems and workflows, such as those focused on designing APIs

These problems might result in certain organizations taking longer to realize the benefits, needing to consider different testing methods, or facing challenges when expanding to very big teams and systems. To address these problems, the PactFlow team expanded the breadth of contract testing capabilities beyond the popular Pact framework to enable new modes of contract capture and compliance, a broader set of use cases, and a simplified developer experience.

11.1.2 How PDCT addresses the drawbacks of CDCT

The core premise of PDCT is that more team members can get involved, using their favorite tools to implement a powerful contract-testing solution with the support of PactFlow. In the following subsections, we discuss how PDCT addresses the drawbacks of CDCT.

CDCT HAS A STEEP LEARNING CURVE

It's no surprise that learning about CDCT has its own challenges. As we first mentioned in chapter 2, section 2.5, CDCT is more technical than other testing activities. The steep learning curve might deter team members from trying out tools such as Pact.